UL2ME007 – Algorithmique et programmation scientifique

TP 3 : Tableaux et masques

Ce TP s'appuie sur le cours 2 et le TD 2 du même nom.

La syntaxe nécessaire est rappelée dans les fiches de cours Python

- 4 - Tableaux numpy - vectorisation
- 5 - Masques

> A partir de ce TP, vous utilisez uniquement l'environnement de développpement (IDE) Spyder. N'oubliez pas d'utiliser tous les outils de Spyder, notamment
>
> - l'**explorateur de variables** pour visualiser les variables en mémoire,
> - la **console Ipython** pour effectuer des opérations simples de vérification.

1 - Manipulation d'un vecteur NumPy : températures au cours du mois

2 - Création et manipulation de matrice

3 - Initalisation de la matrice de Vandermonde

1 - Manipulation d’un vecteur NumPy : températures au cours du mois

> *Note: Cet exercice reprend largement l'exercice 1 du TD2. On ajoute notamment des tracés qui permettent de visualiser les opérations effectués. Pour écrire le code qui réalise ces tracés, vous pouvez reprendre les exemples très similaires de la fiche 5 - Masques.*

Voici un tableau NumPy contenant les températures maximales de chaque journée du mois de juin 2026 à Paris, en $^\circ$C :

```
import numpy as np
import matplotlib.pyplot as plt

Tjuin2026 = np.array([26.4 ,21.7 ,20.2 ,22.0 ,20. ,21.6 ,23.0 ,23.3 ,20.5 ,19.0 ,19.9,\
 26.2 ,28.5 ,26.8 ,28.6 ,29.0 ,32.4 ,36.9 ,36.3 ,35.8 ,36.9,\
 37.7 ,37.4 ,40.6 ,39.8 ,37.3 ,37.2 ,31.3 ,28.8 ,29.6])
```

0. Tracez les températures en fonction du jour du mois (de 1 à 30) dans un graphe dédié. N'oubliez pas la présentation (nom des axes ...).
1. Afficher les températures des 7 premiers jours et des 7 derniers jours.
2. En utilisant des commandes NumPy, afficher la température : (a) moyenne, (b) minimale, (c) maximale, (d) moyenne sur les 15 derniers jours.
3. Construire un masque indiquant les jours où la température dépasse 25°C. Afficher les températures des jours correspondants puis leur nombre.
4. Identifier les jours où la température a été supérieure à la moyenne du mois d'au moins 5°C. Afficher ces jours et les températures de ces journées dites exceptionnellement chaudes.
5. Reprendre le graphe initial dans une nouvelle figure. Indiquez sur ce graphe la moyenne avec une ligne horizontale, et les journées exceptionnelles avec des marqueurs rond rouges. N'oubliez pas la légende.

In [ ]:
```
# YOUR CODE HERE
raise NotImplementedError()
```

In [ ]:
```
# YOUR CODE HERE
raise NotImplementedError()
```

In [ ]:
```
# YOUR CODE HERE
raise NotImplementedError()
```

In [ ]:
```
# YOUR CODE HERE
raise NotImplementedError()
```

2 - Manipulation de matrices [extrait d'examen de TP 2025]

> *Note : on utilise la convention de notation suivante : les composantes d'une matrice M de taille $n \times n$ sont notées M_{ij} avec $1 \leq i, j \leq n$. Attention : c'est la convention mathématique où les indices commencent à 1, alors qu'en Python ils commencent à 0. Il faut donc adapter votre code.*

1. Demander à l'utilisateur d'entrer un entier $n > 2$. La saisie sera reproposée tant que cette condition n'est pas verifiée. Dans la suite on saisira $n = 5$.

2. Créer une matrice $n \times n$ nommée A telle que $A_{ij} = n - |i - j|$. Affichez A.

3. Soit $\alpha = 1/(2n + 2)$. Créer une matrice $n \times n$ nommée B telle que

$$B_{ij} = I_{ij} + \begin{cases} -n\alpha & \text{si } i = j = 1 \text{ ou si } i = j = n \\ -1/2 & \text{si } |i - j| = 1 \\ \alpha & \text{si } |i - j| = n - 1 \\ 0 & \text{sinon} \end{cases}$$

où I est la matrice identité d'ordre n. Affichez B.

4. Calculez la matrice P, produit matriciel de A par B. Affichez P.

5. En utilisant un masque, "nettoyer" la matrice P en mettant à zéro tout élément de valeur absolue inférieure à 10^{-8}. Affichez la nouvelle matrice P.

In [ ]:
```
# YOUR CODE HERE
raise NotImplementedError()
```

3 - Initialisations de la matrice de Vandermonde

> *Note: Cet exercice reprend largement l'exercice 2 du TD2. On ajoute des tests qui permettent de comparer les temps d'exécution des différentes alternatives.*

Soit $X = (x_1, x_2, \ldots, x_n)$ un vecteur de $\mathbb{R}^n$ ($n \geq 2$). On appelle matrice de Vandermonde associée à X la matrice $V[X]$ de taille $n \times n$ définie en notation indicielle par

$$V_{ij}[X] = x_i^{j-1}$$

avec $i = 1, \ldots, n$ les indices de ligne et $j = 1, \ldots, n$ ceux de colonnes. Sous forme développée, la matrice s'écrit donc:

$$V[X] = \begin{bmatrix} 1 & x_1 & x_1^2 & \ldots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \ldots & x_2^{n-1} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_n & x_n^2 & \ldots & x_n^{n-1} \end{bmatrix}$$

1. On donne un début de script Python.

```
import numpy as np
X = np.array([1 ,2 ,3, 4]) #exemple d'initialisation de X
n = np.size(X)
```

Le compléter de telle sorte qu’il initialise la matrice de Vandermonde V composante par composante. Le code doit fonctionner pour un tableau quelconque X de n éléments réels.

> *Note : Attention à la numérotation des éléments d’un tableau python qui commence à l’indice 0, alors que la convention mathématique ci-dessus commence à l'indice 1.*

2. On veut à présent gagner en efficacité et initialiser la matrice en la remplissant *colonne par colonne*. Ecrire le code qui réalise cette implémentation.

3. En utilisant $x_i^j = x_i^{j-1} \times x_i$, proposer une variation de l'initialisation précédente qui évite les calculs de puissances et n'utilise que des mutiplications.

4. Proposer à présent un code qui procède à l’initialisation *ligne par ligne* en utilisant le vecteur des puissances `p = np.arange(n)`.

In [ ]:
```
# YOUR CODE HERE
raise NotImplementedError()
```

On veut à présent comparer ces façons de faire, et pour cela on va utiliser la bibliothèque `timeit` qui permet de mesurer le temps d'exécution d'un bloc de code :

```
import timeit
nit = 50 # nombre de répétitions du bloc

def bloc():
    # ... votre code ici ...
    pass

temps = timeit.timeit(bloc,number=nit) # temps d'exécution moyenné sur nit répétitions

print(f"Temps d'exécution :", temps, " secondes")
```

> *Note : on voit qu'on utilise la syntaxe des fonctions avec le mot-clef `def` pour définir un bloc de code qui sera appelé ensuite. Le chapitre à venir sur les fonctions reviendra plus en détails sur la syntaxe associée.*

6. Complétez ce code pour mesurer le temps d'éxécution des initialisations 1 et 2 ci-dessus.

In [ ]:
```
# YOUR CODE HERE
raise NotImplementedError()
```

Pour initialiser une petite matrice 4×4 comme ci-dessus, toutes les méthodes sont très rapides. Pour bien comparer ces méthodes, on va mesurer ce temps pour des matrices $n \times n$ avec n de plus en plus grand (toujours des matrices de Vandermonde avec `X=np.arange(1,n+1)` pour simplifier). On choisit les 4 valeurs suivantes : $n = 5, 10, 20, 40$. On a donc 4 méthodes à comparer pour 4 valeurs du paramètre n. On va donc stocker les valeurs de temps dans un tableau `temps` 4×4 (chaque ligne correspond à une méthode) pour ensuite afficher les résultats sous forme de graphe.

7. Au moyen d'une boucle sur `n`, calculez les 16 valeurs de temps d'exécution et remplissez le tableau `temps`.

8. Affichez les résultats dans un graphe avec `n` en abscisse, les temps d'exécution en ordonnée, et une courbe pour chaque méthode. Que constatez-vous ? Comment l'expliquez-vous ?

> *Note : comme n varie de façon exponentielle (en puissances de 2), il vaut mieux utiliser un tracé en échelle logarithmique avec `plt.loglog` plutôt que `plt.plot`.*

In [ ]:
```
# YOUR CODE HERE
raise NotImplementedError()
```

In [ ]:
```
# YOUR CODE HERE
raise NotImplementedError()
```

In [ ]:
```
# YOUR CODE HERE
raise NotImplementedError()
```